Project Vulner

Name: Lam Bing Qing Andrew
Student: s5
Unit: CFC060524
Trainer: Samson

Table of Contents:

1. Introduction

2. Script
    2.1. Functions
        - uinIP
        - uinop
        - filemgr
        - brutenlist &bruteplist
    2.2. Get user input for IP address/range and scan type
    2.3. Save directory location
    2.4. netdiscover
    2.5. nmap & masscan
    2.6. Bruteforcing
    2.7. Full scan
    2.8. Zip files to archive
    2.9. Open in geany

1. Introduction

This report aims to explain the accompanying script and what commands and functions were used to achieve its objectives. It also shows how enumerating a network can be automated

The script is configured to ask for user input for an IP address or range to enumerate. Depending on the type of scan chosen, the script would also show potential vulnerabilities of the IP address or range.

2. Script

2.1. Functions

- `uinIP`

```
14  #~ Standard Regex expressions for validation checking of IP address and range
15  rangevalid="^(0?(1?[0-9]{1,2}|2([0-4][0-9]|5[0-5]))\.){3}(0?(1?[0-9]{1,2}|2([0-4][0-9]|5[0-5])))/(0?([1-2]?[0-9]|3[0-2]))$"
16  IPvalid="(0?(1?[0-9]{1,2}|2([0-4][0-9]|5[0-5]))\.){3}(0?(1?[0-9]{1,2}|2([0-4][0-9]|5[0-5])))"
17
18  #~ Function for input of IP address/range
19  function uinIP()
20  {
21      echo
22      read -p 'Key in IP range or address to scan: ' scantar
23
24      #~ Prompting user that input was invalid and looping the function
25      if [[ (! ($scantar =~ $rangevalid)) && (! ($scantar =~ $IPvalid)) ]]; then
26          echo 'Invalid input! Please input a valid IPv4 range or address!'
27          uinIP
28
29      fi
30
31  }
32
```

```
Welcome to Project Vulner!
To use, key in an IPv4 range(eg. 192.168.0.0/24) or address(eg. 192.168.0.1).

Key in IP range or address to scan: 192.168.19.0/24
```

This function is used to ask for user input and save it to variable `scantar`. If the input is invalid(not an IP address or range), the function informs the user and calls on itself to provide a loop until a valid input is detected.

- `uinop`

```
33  #~ Function for input of type of scan
34  function uinop()
35  {
36      echo
37      echo 'Please choose an option: '
38      echo '1. Full scan'
39      echo '2. Basic scan'
40      read -p 'Select 1 or 2: ' option
41
42      #~ Loops if invalid input
43      if [[ ! ($option =~ [1-2]) ]]; then
44          echo 'Invalid input! Please select a valid option!'
45          uinop
46
47      fi
48
49  }
```

```
Please choose an option:
1. Full scan
2. Basic scan
Select 1 or 2: 1
```

This function is used to ask for user input and save it to the variable `option`. If the input is invalid(not 1 or 2), the function informs the user and calls on itself to provide a loop until a valid input is detected.

- `filemgr`

```
51  #~ Function to write into the result files as well as saving files to be zipped
52  function filemgr()
53  {
54      #~ Different variables for netdiscover and the rest
55      if [ "$2" == "netdiscover" ]; then
56          echo "$2 scan results on $scantar: " >> "full_results_$resultname"
57          echo '' >> "full_results_$resultname"
58          cat "$1_$resultname" >> "full_results_$resultname"
59          echo '' >> "full_results_$resultname"
60          echo '' >> "full_results_$resultname"
61          echo
62          echo "$2 scan done on $scantar! Results saved to $savepath/$1_$resultname!"
63
64          zipfiles="$zipfiles $1_$resultname"
65
66      else
67          echo "$2 scan results on $scanIP: " >> "full_results_$resultname"
68          echo '' >> "full_results_$resultname"
69          cat "$1_$scanIP" >> "full_results_$resultname"
70          echo '' >> "full_results_$resultname"
71          echo '' >> "full_results_$resultname"
72          echo
73          echo "$2 scan done on $scanIP! Results saved to $savepath/$1_$scanIP!"
74
75          zipfiles="$zipfiles $1_$scanIP"
76
77      fi
78
79  }
```

`Filemgr` is used to write the results into files as well as saving a list of all the files it has created so the user can have the option to add all the files to a zip archive.

- `brutenlist` & `bruteplist`

```
81  #~ Function to read and validate user provided name list
82  function brutenlist
83  {
84      read -p "Please input your username list file name/path: " unamelst
85
86
87      if !(test -f $unamelst); then
88          echo
89          echo "Username list file name/path does not exist! Please input a valid username list file!"
90
91          brutenlist
92
93      fi
94
95  }
96
97  #~ Function to read and validate user provided password list
98  function bruteplist
99  {
100     read -p "Please input your password list file name/path: " passlst
101
102     if !(test -f $passlst); then
103         echo
104         echo "Password list file name/path does not exist! Please input a valid password list file!"
105
106         bruteplist
107
108     fi
109
110 }
```

```
192.168.19.132 has services that can be bruteforced.
Default list is /usr/share/seclists/Usernames/top-usernames-shortlist.txt.
Would you like to use your own username and password list?(y/n): n
Proceeding with bruteforce ...
```

These two functions have similar features, which are to ask for user input to provide a username or password list file name/path and validate the existence of the provided file name/path. The reason they are two functions is so that the user does not have to reenter both file names/paths if they get one wrong.

2.2. Get user input for IP address/range and scan type

```
112  #~ Script starts here
113  echo 'Welcome to Project Vulner!'
114  echo 'To use, key in an IPv4 range(eg. 192.168.0.0/24) or address(eg. 192.168.0.1).'
115
116  uinIP
117  uinop
```

The script begins by getting the user input through the functions `uninIP`, `uinop` and saving the inputs to variables.

2.3. Save directory location

```
119  #~ Save directory location
120  echo
121  read -p "Please specify a save directory for the output files (Default is the current directory): " savepath
122
123  #~ Setting save location to default directory if no input
124  if [ -z $savepath ]; then
125      savepath='.'
126
127  fi
128
129  #~ Testing for and creating save location if needed
130  if !(test -d $savepath); then
131      echo 'Directory path does not exist! It will be created.'
132      mkdir -p $savepath
133
134  fi
135
136  cd $savepath
```

```
Please specify a save directory for the output files (Default is the current directory):
```

Next the script gets the user input for the save directory location. If no input is given, the save location is set to the current directory. If the input is not a valid directory, it is created.

2.4. `netdiscover`

```
139  #~ netdiscover
140  if [[ $scantar =~ $rangevalid ]]; then
141      resultname=$(echo $scantar | tr '/' '-')
142
143      echo
144      echo 'Proceeding with netdiscover...'
145      sudo netdiscover -r "$scantar" -P > "netdiscover_$resultname"
146      filemgr netdiscover "netdiscover"
147
148      #~ Setting IP address(es) to be scanned
149      echo
150      echo 'Proceeding to scan all available IP addresses in range...'
151      target=$(cat "netdiscover_$resultname" | grep -E "$IPvalid" | awk '{print $1}')
152
153  else
154      resultname="$scantar"
155      target="$scantar"
156
157  fi
```

```
Proceeding with netdiscover ...
[sudo] password for kali:

netdiscover scan done on 192.168.19.0/24! Results saved to ./netdiscover_192.168.19.0-24!

Proceeding to scan all available IP addresses in range ...
```

If the provided input is an IP range, the script proceeds to do a `netdiscover` to find out all the available IP addresses in the range, and saves the results to the full results file and an output file. If only one IP address is provided, the script moves on.

2.5. `nmap` & `masscan`

```
159  #~ Loop for each IP address in network range
160  for scanIP in $target; do
161      #~ nmap TCP scan
162      echo
163      echo
164      echo "Starting nmap TCP scan on $scanIP..."
165      echo
166      sudo nmap $scanIP -sV -Pn -A -oN "nmap_TCP_$scanIP"
167      filemgr nmap_TCP "nmap TCP"
168
169      #~ masscan UDP scan first
170      echo
171      echo
172      echo "Starting masscan UDP scan on $scanIP..."
173      echo
174      sudo masscan $scanIP --rate=10000 -pU:0-65535 -oL "masscan_UDP_$scanIP"
175      filemgr masscan_UDP "masscan UDP"
176
177      UDPresults=$(cat "masscan_UDP_$scanIP" | awk '{print $3}' | grep . | tr '\n' ',')
178
179      #~ if masscan shows results, enumerate UDP ports with nmap
180      if [ -n "$UDPresults" ]; then
181          echo
182          echo
183          echo "Open UDP ports detected. Enumerating open UDP ports."
184          echo "Starting nmap UDP scan on $scanIP..."
185          echo
186          sudo nmap $scanIP -sV -sU -p $UDPresults -A -oN "nmap_UDP_$scanIP"
187          filemgr nmap_UDP "nmap UDP"
188
189      fi
```

```
nmap TCP scan done on 192.168.19.2! Results saved to ./nmap_TCP_192.168.19.2!


Starting masscan UDP scan on 192.168.19.2 ...

Starting masscan 1.3.2 (http://bit.ly/14GZzcT) at 2024-10-13 00:52:32 GMT
Initiating SYN Stealth Scan
Scanning 1 hosts [65536 ports/host]

masscan UDP scan done on 192.168.19.2! Results saved to ./masscan_UDP_192.168.19.2!


Open UDP ports detected. Enumerating open UDP ports.
Starting nmap UDP scan on 192.168.19.2 ...
```

The script then proceeds to do a `nmap` TCP and `masscan` UDP scan on all the IP addresses. If a UDP port is detected, the script then further enumerates that port with `nmap`.

2.6. Bruteforcing

```
#~ Detecting telnet, FTP, RDP or SSH services
loginser='telnet ftp rdp ssh'
brute=''

for ser in $loginser; do
    if [[ $(cat "nmap_TCP_$scanIP" | grep -i $ser) ]]; then
        brute=$ser;

    fi

done

#~ Allow user to set their own username and password list
if [ -n "$brute" ]; then
    echo
    echo
    echo "$scanIP has services that can be bruteforced."
    echo "Default list is /usr/share/seclists/Usernames/top-usernames-shortlist.txt."
    read -p "Would you like to use your own username and password list?(y/n): " bruteop

    if [ "$bruteop" == "y" ]; then
        brutenlist
        bruteplist

    else
        unamelst="/usr/share/seclists/Usernames/top-usernames-shortlist.txt"
        passlst="/usr/share/seclists/Usernames/top-usernames-shortlist.txt"

    fi

    #~ Bruteforce with medusa. Best to be used with SSH. Metasploitable's telnet interface isn't recognised by the tool, FTP allows anonymous logins
    echo "Proceeding with bruteforce..."
    echo
    medusa -h "$scanIP" -U "$unamelst" -P "$passlst" -M "$brute" -t 32 -v 4 -w 0 -O "medusa_$scanIP"
    filemgr medusa "Bruteforce"

fi
```

```
192.168.19.132 has services that can be bruteforced.
Default list is /usr/share/seclists/Usernames/top-usernames-shortlist.txt.
Would you like to use your own username and password list?(y/n): n
Proceeding with bruteforce...

Medusa v2.2 [http://www.foofus.net] (C) JoMo-Kun / Foofus Networks <jmk@foofus.net>

ACCOUNT FOUND: [ssh] Host: 192.168.19.132 User: user Password: user [SUCCESS]

Bruteforce scan done on 192.168.19.132! Results saved to ./medusa_192.168.19.132!
```

If the script detects Telnet, FTP, RDP or SSH services running, it would proceed to do a bruteforce using `medusa`. The user is given the option of providing their own username and password lists. The result is then saved into the full results and output file.

2.7. Full scan

```
229     #~ Full scan
230     if [ $option == 1 ]; then
231         echo
232         echo
233         echo "Full scan selected. Proceeding with vulnerability assessment."
234
235         cat nmap_*_$scanIP | grep -w "open" >> serlist
236
237         #~ I opted to do this instead of searchsploit --nmap because --nmap's output would be a whole lot longer
238         #~ Also I can make the output neater this way
239         cat serlist | while read -r serv; do
240             serport=$(echo $serv | awk '{print $1}')
241             serscan=$(echo $serv | awk '{print substr($0, index($0,$3))}')
242
243             echo "searchsploit vulnerability assessment for $serport $serscan: " >> "searchsploit_$scanIP"
244             searchsploit $serscan >> "searchsploit_$scanIP"
245             echo '' >> "searchsploit_$scanIP"
246
247         done
248
249         #~ Writing results to output file
250         echo '' >> "searchsploit_$scanIP"
251         echo
252         echo "searchsploit vulnerability assessment done! Results saved to $savepath/searchsploit_$scanIP!"
253
254         #~ Writing results to full results file
255         echo "searchsploit vulnerability assessment result for $scanIP: " >> "full_results_$resultname"
256         echo '' >> "full_results_$resultname"
257         cat "searchsploit_$scanIP" >> "full_results_$resultname"
258         echo '' >> "full_results_$resultname"
259         echo '' >> "full_results_$resultname"
260
261         zipfiles="$zipfiles searchsploit_$scanIP"
262         rm serlist
263
264     fi
```

```
Full scan selected. Proceeding with vulnerability assessment.

searchsploit vulnerability assessment done! Results saved to ./searchsploit_192.168.19.132!
```

If the full scan option is selected, the script would run all the found services through `searchsploit` to determine potential vulnerabilities. It then saves the results.

2.8. Zip files to archive

```
270   #~ Zip files to archive
271   echo
272   read -p "Would you like to zip all result files into an archive?(y/n) " zipf
273
274   if [ "$zipf" == "y" ]; then
275       zip "Vulner_$resultname.zip" $zipfiles
276
277   fi
278
```

```
Would you like to zip all result files into an archive?(y/n) y
        zip warning: name not matched: full_results_
  adding: netdiscover_192.168.19.0-24 (deflated 68%)
  adding: nmap_TCP_192.168.19.1 (deflated 36%)
  adding: masscan_UDP_192.168.19.1 (stored 0%)
  adding: searchsploit_192.168.19.1 (stored 0%)
  adding: nmap_TCP_192.168.19.2 (deflated 37%)
  adding: masscan_UDP_192.168.19.2 (stored 0%)
  adding: nmap_UDP_192.168.19.2 (deflated 36%)
  adding: searchsploit_192.168.19.2 (deflated 64%)
  adding: nmap_TCP_192.168.19.128 (deflated 39%)
  adding: masscan_UDP_192.168.19.128 (stored 0%)
  adding: medusa_192.168.19.128 (deflated 44%)
  adding: searchsploit_192.168.19.128 (deflated 59%)
  adding: nmap_TCP_192.168.19.131 (deflated 36%)
  adding: masscan_UDP_192.168.19.131 (stored 0%)
  adding: searchsploit_192.168.19.131 (stored 0%)
  adding: nmap_TCP_192.168.19.132 (deflated 54%)
  adding: masscan_UDP_192.168.19.132 (deflated 32%)
  adding: nmap_UDP_192.168.19.132 (deflated 40%)
  adding: medusa_192.168.19.132 (deflated 42%)
  adding: searchsploit_192.168.19.132 (deflated 86%)
  adding: nmap_TCP_192.168.19.254 (deflated 36%)
  adding: masscan_UDP_192.168.19.254 (stored 0%)
  adding: searchsploit_192.168.19.254 (stored 0%)
```

The user is then given the option to add all the result files into a ZIP archive.

2.9. Open in geany

```
279   #~ Open in geany
280   echo
281   read -p "Would you like to open $savepath/full_results_$resultname in geany?(y/n) " geanyfile
282
283   if [ "$geanyfile" == "y" ]; then
284       geany "full_results_$resultname"
285
286   fi
287
```

```
Would you like to open ./full_results_192.168.19.0-24 in geany?(y/n) y
```

At the end the user is prompted for whether they want to open the full results file in geany to do their own searching.